

Patient Intake & Symptom Triage Assistant

I built this to get hands-on with something I kept saying I wanted to learn: wiring an LLM into a real backend service, not just calling an API and printing the response, but actually using it as part of the business logic.

The idea is simple — a patient submits a short description of their symptoms, and the system automatically figures out which department it should go to and how urgent it is. No manual triage step, no separate "click to classify" button. You create the note, it comes back already classified.

■ **This uses fake data only.** *There's no login, no encryption, no audit trail — none of the things a real healthcare system would need. I built this to learn the AI + backend integration pattern, not to handle actual patient information, and it should never be used that way.*

What it actually does

You send a patient ID, age, and a symptom description like "sudden crushing chest pain, sweating heavily, pain spreading to jaw." Behind the scenes:

- The note gets saved to the database
- The symptom text gets sent to a locally-running LLM
- The model reads it and returns a department (Cardiology, Dermatology, etc.) and an urgency level
- That comes back to you in the same response — the whole thing happens in one API call

I wanted this to feel like something a backend engineer would actually build, not a toy chatbot demo — so the AI part is just one step inside a normal REST + database flow, the same way you'd call any other external service.

How it's put together

```
Client (Swagger UI / curl)
  |
  v
Spring Boot Controller
  |
  +--> Database (H2)
  |
  +--> Ollama (local LLM)
```

(Diagrams omitted from this PDF version — see the Markdown version of this doc for the sequence and component diagrams, which render as visuals on GitHub.)

Tech stack

- **Java 17 / Spring Boot** — for the REST API and database layer
- **H2** — in-memory database, easy to run without installing anything
- **Ollama running llama3.2:1b** — the LLM doing the classification

- **springdoc + Swagger UI** — so I could actually test endpoints without needing IntelliJ Ultimate's HTTP client
 - **Maven** — build tool
-

Why I used Ollama instead of a paid API

Honestly, this was mostly a practical decision. I didn't want to sign up for anything or deal with an API key while I was still figuring out how the whole pipeline should work. Ollama let me run a real model locally, for free, and just get on with building.

What worked well about it

- No cost, no rate limits — I could hit it as many times as I wanted while testing
- No API key setup, no cloud account — installed it and it just worked
- Everything stays on my own machine, which felt right even for fake data
- The code pattern (send text, get structured JSON back) is exactly the same pattern you'd use for Claude or GPT-4 — swapping providers later is a small, contained change, not a rewrite

What it's not great at

- It's a small model (1 billion parameters), so its judgment isn't as sharp as a bigger model's. I actually caught this myself while testing — I sent in a textbook description of a heart attack and it came back with LOW urgency, which is clearly wrong. That's not a bug in my code — the pipeline did exactly what it's supposed to do — it's just the model itself not being strong enough for a nuanced medical judgment call.
- You need Ollama installed locally to run this at all, which adds a setup step a hosted API wouldn't have
- It's slower than a cloud API since it's running on a regular laptop CPU

I think that trade-off is actually worth talking about, rather than hiding — knowing why you picked a smaller model, and being honest about what it costs you in accuracy, feels like a more useful thing to understand than just always reaching for the biggest model available.

The data model

The main entity is **IntakeNote** — it holds the patient's info, the symptom text, and (once classified) the department and urgency.

I also added a separate **CreateIntakeNoteRequest** class for what a client actually sends in. At first I just used the same **IntakeNote** class for both incoming requests and outgoing responses, but that meant Swagger's auto-generated example showed fields like `id`, `status`, and `department` as if the client was supposed to fill them in — which isn't true, since the server controls all of that. Splitting it into a separate request class fixed that: now the API only asks for what it actually needs — `patientId`, `patientAge`, and `symptomText`.

One more thing worth calling out: `id` and `patientId` are two different things. `id` is just the database row number — it means nothing outside this table. `patientId` is meant to represent the actual person, so the same patient can have multiple notes over time, each with its own `id`, all sharing the same `patientId`.

Endpoints

Method	Path	What it does
POST	/intake-notes	Create a note — gets classified automatically
GET	/intake-notes	List everything
GET	/intake-notes/{id}	Get one note
PUT	/intake-notes/{id}	Update a note
DELETE	/intake-notes/{id}	Delete a note
POST	/intake-notes/{id}/classify	Re-run classification (e.g. after editing symptom text)

Example — creating a note

POST /intake-notes
Content-Type: application/json

```
{
  "patientId": "P-3001",
  "patientAge": 60,
  "symptomText": "Sudden crushing chest pain, sweating heavily, pain spreading to jaw"
}
```

What comes back

```
{
  "id": 16,
  "patientId": "P-3001",
  "patientAge": 60,
  "symptomText": "Sudden crushing chest pain, sweating heavily, pain spreading to jaw",
  "status": "SUBMITTED",
  "createdAt": "2026-09-06T15:40:32.225",
  "department": "Cardiology",
  "urgency": "LOW"
}
```

(That LOW urgency is the exact model limitation I mentioned above — department came back right, urgency didn't. Left it in as an honest example rather than cherry-picking a cleaner result.)

How to actually run this

1. Install Ollama

Go to ollama.com, download it, install it like any normal app. When it asks you to sign up, you can skip that — click "No thanks, I'll use Ollama locally." You don't need an account for this.

Then pull the model:

```
ollama pull llama3.2:1b
```

Quick sanity check that it's working:

```
ollama run llama3.2:1b
```

Type something, see if it replies. Type /bye to leave. Ollama keeps running in the background after that — you don't need to keep this window open.

2. Run the app

Open the project in your IDE (make sure you open the folder that actually has pom.xml in it, not a parent folder — I made this mistake myself and ended up with a bunch of unrelated projects cluttering the sidebar).

Let Maven grab the dependencies, then run IntakeApiApplication. You'll know it worked when you see "Started IntakeApiApplication" in the console. That also seeds the database with 15 fake intake notes automatically, so there's data to look at right away.

3. Peek at the database (optional)

Go to <http://localhost:8080/h2-console> while the app's running. JDBC URL needs to be `jdbc:h2:mem:intakedb` — not the default `testdb` it suggests, that'll fail. Username `sa`, no password. Then:

```
SELECT * FROM intake_notes;
```

4. Test the API

I ended up using Swagger UI for this, mainly because IntelliJ Community edition doesn't include the built-in HTTP client, and I didn't want to fight with Postman or curl's quoting rules in PowerShell (that took way longer to figure out than it should have).

```
http://localhost:8080/swagger-ui/index.html
```

Expand POST /intake-notes, hit "Try it out," paste in a symptom description, hit Execute. Give it 10-30 seconds on the first request — the model has to load.

If you'd rather use curl and you're on Windows, save the JSON to a file first instead of trying to inline it — PowerShell mangles escaped quotes:

```
$json = '{"patientId":"P-3001","patientAge":60,"symptomText":"..."}'  
$json | Out-File -Encoding utf8 body.json  
curl.exe -X POST http://localhost:8080/intake-notes -H "Content-Type: application/json" -d "@body.json"
```

On Mac/Linux, inline JSON just works normally with curl.

What I'd still want to fix

- **Accuracy.** The small model's judgment isn't reliable enough for anything real. Swapping in Claude or GPT-4 would fix this — and since the classification logic is isolated in its own service class, that'd be a contained change, not a rewrite.
- **No real persistence.** H2 wipes on every restart. Fine for messing around, not fine for anything real — Postgres would fix that.
- **No similarity search yet.** I'd like to add pgvector so new notes get compared against past ones, which could make the classification smarter by giving it more context.

- Obviously — auth, encryption, all the things you'd actually need before this touched real data.
-

Why I built it this way

Mostly to prove to myself I could actually wire an LLM into a normal backend service, end to end — not just hit an API and print the result, but have it be a real step in the business logic, with its output flowing into the database and back out through the API like anything else.

It also forced me to think about things I hadn't really had to before — what happens when the model gives a wrong answer, how to design a prompt that reliably returns parseable JSON, and where to draw the line between "the code is working correctly" and "the model's judgment isn't good enough." That distinction turned out to be one of the more useful things I took away from building this.